NeuroSync - Project Report

Evidence-driven, multi-agent investigation workspace on a Blackboard Architecture

Version 1.0.0

Generated for internal reference

FastAPI - SQLAlchemy 2.0 - React 19 - TypeScript

NeuroSync - Complete Project Report

An evidence-driven, multi-agent investigation workspace built on a Blackboard Architecture

- Project: NeuroSync - AI-powered multi-agent conflict detection & investigation platform
- Version: 1.0.0
- Report scope: from project start to the latest upgrade, with run instructions, innovation, and an anticipated external Q&A
- Audience: internal reference, evaluation, and external review

---

Table of contents

1. Executive summary
2. The problem NeuroSync solves
3. What NeuroSync is
4. Innovation - what makes this different
5. The development journey (start to finish)
6. Architecture at a glance
7. The blackboard data model
8. The six agents
9. The dynamic scheduler
10. Conflict detection rules
11. Evidence & hypothesis scoring
12. Multi-factor evidence assessment (upgrade)
13. Human review, audit trail & decisions (upgrade)
14. Dashboard filters & distributions (upgrade)
15. Reports
16. Knowledge Graph
17. Optional AI-assisted reasoning & safety
18. Security, configuration & secrets
19. Technology stack
20. REST API surface
21. Frontend application
22. Testing strategy
23. Evaluation results
24. Run instructions (local)
25. Docker & deployment
26. Known limitations
27. Anticipated external questions & answers
28. Glossary
25. Credits & license

---

1. Executive summary

NeuroSync ingests documents, extracts structured claims, detects contradictions across sources, and then lets a collective of six specialised reasoning agents investigate those conflicts on a shared blackboard. Agents never call each other; a dynamic scheduler decides, each cycle, which agent should act next based on severity, uncertainty, evidence gaps and expected information gain. Every action,

evidence item and conclusion is transparent, traceable and auditable.

NeuroSync runs fully without any AI API key, using deterministic rule-based reasoning so results are reproducible. An optional, validated OpenAI-compatible LLM layer can be enabled at runtime for extra recall and natural-language explanations - but the system is correct and complete without it.

The project reached a verified 1.0.0 release state and was then extended with a focused upgrade (evidence assessment, richer human review, and dashboard analytics). The backend test suite is 216 tests, all passing, and the production frontend build succeeds.

---

2. The problem NeuroSync solves

Organisations hold the same facts in many documents - inspection reports, maintenance logs, sensor records, spreadsheets - and those sources routinely disagree. Today the work of noticing a disagreement, gathering the evidence that explains it, weighing competing explanations, and recording a defensible decision is manual, slow and often undocumented.

Three problems follow:

1. Detection is inconsistent. Contradictions are missed or, worse, invented from unrelated numbers.
2. Reasoning is a black box. When a tool offers an answer, you cannot see which evidence drove it.
3. Accountability is absent. There is no audit trail, no explicit uncertainty, and no human sign-off.

NeuroSync addresses all three with a transparent, traceable pipeline that is useful even with no machine-learning model - and safer when one is added.

---

3. What NeuroSync is

NeuroSync is a full-stack web application:

- Backend: FastAPI + SQLAlchemy 2.0 + SQLite (default) + Pydantic v2.
- Frontend: React 19 + TypeScript + Vite + Tailwind CSS v4, with React Flow (knowledge graph) and Recharts (dashboard charts).
- Realtime: Server-Sent Events (SSE) stream each agent action live.
- Reasoning: six specialised agents coordinated by a dynamic scheduler over a shared blackboard.
- Human-in-the-loop: reviewers approve, reject, mark inconclusive, request more investigation, or reopen a conflict; every decision is logged.

The headline capabilities:

- Blackboard pipeline: documents -> claims -> conflicts -> evidence -> hypotheses -> conclusions.
- Deterministic, explainable conflict detection with a human-readable "why was this flagged?".
- Evidence weighting and explainable hypothesis ranking.
- Explicit uncertainty handling (missing evidence is surfaced, not hidden).
- Optional, validated AI assistance.
- Realtime experience, exportable reports, and a Knowledge Graph.
- A clearly labelled demonstration scenario.

---

4. Innovation - what makes this different

NeuroSync's contribution is not "another chatbot over your documents". Its innovations are architectural and epistemic:

1. Blackboard multi-agent architecture, not a pipeline. Agents do not call each other. They independently read the shared blackboard, decide whether they have useful work, and write new knowledge. This makes the system extensible (add an agent, not a stage) and emergent (the order of work adapts to the data).
2. A dynamic, explainable scheduler. Instead of a fixed workflow, each cycle scores every candidate action by severity, uncertainty, evidence gap and expected information gain. The chosen action, its factor breakdown, and the state it was derived from are all logged - so *why the system did what it did* is always answerable.
3. Deterministic-first, AI-optional reasoning. The entire pipeline works with zero credentials using transparent rules, so results are reproducible and auditable. A language model is an *optional accelerator*, never a dependency.
4. Untrusted-model safety by construction. When AI is enabled, document text is passed as *data* (prompt-injection hardened), any supporting quote must appear verbatim in the source, and any citation that does not resolve to a real evidence row is discarded. AI suggestions are tagged so they are always distinguishable from rule-based output.
5. Uncertainty as a first-class output. The system refuses to force a conclusion. Missing information, unresolved questions and limitations are surfaced explicitly rather than hidden behind a single confidence number.
6. Full provenance and auditability. Every claim, evidence item, hypothesis, verification, conflict, conclusion and review carries its database id, and every blackboard change is written to an append-only audit trail with an actor and timestamp.
7. Inspectable, multi-factor evidence assessment (upgrade). Each evidence item is scored on relevance, provenance, recency, completeness and corroboration - with per-factor labels, explanations, a documented formula and an explicit "unknown" source-reliability label instead of a fabricated number.
8. A labelled evaluation harness. Reliability is measured against a curated, versioned dataset, with the failure cases retained and documented - the project reports what it cannot do as clearly as what it can.

In one line: an auditable, deterministic-by-default multi-agent reasoning system that treats transparency and uncertainty as features, not afterthoughts.

---

5. The development journey (start to finish)

NeuroSync was developed in three broad phases. This section records what was built and verified, in order.

Phase A - Core platform

The blackboard foundation and all core features were implemented and covered by tests:

- Document ingestion and text extraction (PDF via PyMuPDF, DOCX via python-docx, and text formats).
- Rule-based claim extraction (subject, predicate, value, unit, polarity, time).
- Deterministic conflict detection and classification.

- Evidence collection, hypothesis generation and verification scoring.
- The Decision Agent and conservative decision thresholds.
- The dynamic scheduler and the orchestrator execution ledger.
- Realtime SSE events with redaction and replay.
- Explainability services: hypothesis ranking, audit trail, human review.
- Report generation (Markdown, JSON, PDF).
- The optional, validated LLM layer.
- The knowledge-graph projection and its UI.
- Security controls (bearer-token gating, upload validation, secret redaction).
- The React frontend (dashboard, conflicts, evidence, hypotheses, investigations, knowledge graph, reports, audit, settings, agents).

Phase B - Release preparation and verification (v1.0.0)

A full verification pass produced the release artifacts. Measured results:

#	Check	Result
1	Full backend test suite	204 passed, 0 failed (147.31s, 6 warnings)
2	Frontend production build	Succeeded (2360 modules, 4.24s)
3	Evaluation dataset	Reproduced exactly (see Sec 23)
4	Six-agent workflow	6/6 agents acted; investigation completed
5	Evidence traceability / uncertainty	Traceable; explicit uncertainty surfaced
6	Knowledge Graph	23 nodes, 40 edges; all required relations present
7	Report generation	28 sections; Markdown + JSON + PDF all render
8	Configuration / secret handling	No committed secrets; redaction active
9	DB persistence / backup	SQLite on a persistent volume; backup/restore scripts present
10	Prod frontend/backend config	Coherent (token injected server-side, single worker)
11	App starts via documented commands	Backend 200 (health/live/root/documents); frontend 200
12	README / installation guide	Reviewed - accurate

Deliverables produced: CHANGELOG.md, docs/release_notes.md, docs/release_checklist.md, docs/known_issues.md, and README updates.

Phase C - Professional upgrade

Rather than rebuild, the mature codebase was inspected and a formal gap analysis (docs/FEATURE_GAP_ANALYSIS.md) was written, section by section, against the requested feature set. Seven of the requested feature areas were already fully implemented; four genuine gaps were identified and closed:

- G1 - Multi-factor evidence assessment (Sec 4): new services/assessment_service.py; exposes an assessment object on every evidence item; rendered on the Evidence page.
- G2 - Review decisions inconclusive and reopen (Sec 6): extended ReviewDecision, review_service mappings and schemas; added UI buttons; every review reports review_kind = "local_manual_review".
- G3 - Dashboard filters & distributions (Sec 9): the blackboard snapshot gained a days window and four enum distributions; the dashboard gained a 7/14/30/90 day selector and clickable distribution cards that deep-link into pre-filtered list pages.
- G4 - Documentation (Sec 17): docs/ARCHITECTURE.md, docs/API_REFERENCE.md, docs/TESTING.md, docs/FEATURE_GAP_ANALYSIS.md.

Phase C verification

- Backend suite grew to 216 passing (+12 tests: a new evidence-assessment

module, a new dashboard-snapshot module, and new review-decision cases).

- Frontend production build succeeds.
- An HTTP-level smoke test of the live app was run end-to-end and caught a real defect: GET /api/blackboard/snapshot?days=7 returned window_days: 14 because the handler never set the field (the schema default won). It was fixed and the snapshot tests were strengthened to assert window_days.
- No database migration was required for any upgrade (assessment is computed on read; review reuses the free-text decision column; the snapshot adds no columns).

Final verified state: backend 216 passed / 0 failed; frontend build green; smoke test OK.

---

6. Architecture at a glance

```
                +----------------------- Blackboard -----------------------+
                |  documents - claims - conflicts - evidence - hypotheses     |
                |  conclusions - activity log - events                        |
                +-------^--------------^--------------^--------------^----+
                        | read/write   |              |              |
   Evidence Collector ---+   Conflict Detector   Hypothesis Generator   Investigator
                                                    Verification Agent   Decision Agent
                        ^
                Dynamic Scheduler (severity - uncertainty - evidence gap - info gain)
```

Design principles:

1. The blackboard is the only shared state. Agents never call each other.
2. Opportunistic control. No fixed workflow; the scheduler chooses each step.
3. Deterministic by default. Fully reproducible without any API key.
4. No forced conclusions. If evidence is insufficient, the system says so.
5. Every decision is explained. Activities, reasons and rationales persist.

Data flow: Ingest -> Detect -> Hypothesize -> Investigate -> Verify -> Decide -> Report. Steps 2-6 are driven by the scheduler, not a hard-coded order.

---

7. The blackboard data model

The blackboard is a relational database (SQLite by default). Core entities:

Entity	Meaning
Document	An uploaded or seeded source plus extracted text.
Observation	A raw sentence-level observation from a document.
Claim	A structured assertion (subject, predicate, value, unit, polarity, time).
Conflict	A detected contradiction between two claims, with type and severity.
Evidence	Anything that bears on a conflict (origin/reliability/directness).
Hypothesis	A candidate explanation for a conflict, later scored.
Conclusion	The Decision Agent's final assessment for a conflict.
AgentActivity	An audit log of every agent action, reason and blackboard delta.
EventLog	An append-only event stream used by the SSE endpoint.
AuditEvent	Append-only record of every blackboard change, with actor.
HumanReview	A human decision on a conclusion, with reviewer and note.
AppSetting	Persisted application settings (e.g. reasoning mode).

The provenance chain:

```
Document --> Observation --> Claim --+
                                     +--> Conflict --> Evidence --> Hypothesis --> Conclusion
Document --> Observation --> Claim --+
```

8. The six agents

All agents derive from BaseAgent and implement candidates() (available work) and run(). Each declares a contract (required_inputs, expected_outputs, run_conditions, stop_conditions), surfaced in the UI.

Agent	Responsibility
Evidence Collector	Extract claims and observations from documents.
Conflict Detector	Compare cross-document claims and flag contradictions.
Hypothesis Generator	Propose competing explanations for each conflict.
Investigator	Gather additional relevant evidence from the documents.
Verification Agent	Score hypotheses from the evidence; validate LLM explanations.
Decision Agent	Synthesise a conclusion and set the final conflict status.

Agents never communicate directly - everything flows through the blackboard.

9. The dynamic scheduler

Each cycle, the scheduler scores every candidate action and runs the most valuable one:

```
priority = 100 - (0.30-severity + 0.25-uncertainty + 0.25-evidence_gap + 0.20-info_gain)
```

The chosen action, its factor breakdown, its numeric priority, and the BlackboardState it was derived from are recorded on the resulting AgentActivity and streamed as an event. Execution uses a ledger (sequence, attempt, input/output record ids, error, state summary) so runs are fully auditable, with retries, attempt caps and deterministic stop conditions (no candidates, priority below threshold, iteration limit, execution limit, cancel, or completion).

10. Conflict detection rules

Detection is deterministic and conservative. It only compares claims sharing a property (the predicate, falling back to subject) from different documents, and guards against false conflicts between unrelated quantities.

Type	Trigger	Severity
direct_contradiction	Same predicate, opposite polarity	high
numerical_disagreement	Same unit, values differ > 2% with no distinguishing context	medium/high
temporal_inconsistency	Values differ but timestamps > 30 min apart	medium
missing_context	Units differ/missing, or only one side has a timestamp	low/medium
source_disagreement	Sources disagree on a shared assertion	medium
semantic_inconsistency	Truthy states differ in wording	medium

A relative difference <= 2% is treated as agreement; duplicate conflicts are never created. Each conflict records the matched subject, property and reason.

11. Evidence & hypothesis scoring

Scores are bounded support indicators, not calibrated probabilities.

Single evidence weight:

```
weight = 0.40-relevance + 0.35-reliability + 0.25-directness
```

Baseline reliability by origin: document 0.60, user 0.55, inferred 0.35. Hypothesis support sums positive weights (down-weighting repeated evidence from the same document) and subtracts negative weights,

then soft-saturates into [0, 1]. Status thresholds: contradicted, supported, review_required or inconclusive, with a single-source supported downgraded to review_required.

The Decision Agent then compares the top two hypotheses and applies conservative thresholds, producing supported_conclusion, tentative_conclusion, human_review_required or inconclusive, and updating the conflict status.

---

12. Multi-factor evidence assessment (upgrade)

Every evidence item now carries an inspectable assessment object built by services/assessment_service.py:

```
overall = 0.35-relevance + 0.25-provenance + 0.15-recency
        + 0.15-completeness + 0.10-corroboration
overall x= 0.85   when contradictory evidence exists
```

- relevance - overlap between the evidence text and the claim it concerns.
- provenance - origin/type reliability (document > user > inferred).
- recency - freshness of the *source* timestamp; labelled unknown when no source timestamp exists (the record's own creation time is deliberately not used).
- completeness - how many expected fields are present.
- corroboration - how many distinct sources agree.

The object also includes per-factor labels and explanations, a source_reliability label (explicitly "unknown" rather than a fabricated value), a missing_information list, the formula, and a caveat. It is rendered on the Evidence page.

---

13. Human review, audit trail & decisions (upgrade)

A reviewer can record a decision on any conclusion:

Decision	Effect on conflict
approved	-> resolved
rejected	-> open
inconclusive	-> inconclusive
request_more_investigation	-> human_review
reopen	-> open (re-open a previously reviewed conflict)

The human_reviews row stores the decision, reviewer, note, the conclusion's previous decision type and a timestamp, and reports review_kind = "local_manual_review". The decision is written to the audit trail and published as a human_review_recorded event. The audit trail is an append-only table recording every blackboard change with actor and time. Hypotheses are additionally ranked by documented evidence weight, with the supporting/contradicting evidence, criteria, assumptions, missing information and margin to the next explanation.

---

14. Dashboard filters & distributions (upgrade)

The Overview dashboard is driven entirely by GET /api/blackboard/snapshot, which reads persisted rows. It now accepts a days window (default 14, range 1-365) and returns four enum distributions - conflict type, conflict status, investigation status and evidence verification - each listing every known value (including zero counts) for stable UI options. The dashboard adds a 7/14/30/90 day selector and clickable distribution cards that deep-link into pre-filtered list pages (/conflicts?type=..., /conflicts?status=..., /investigations?status=..., /evidence?verification_status=...).

15. Reports

services/report_service.py assembles a report only from persisted records and returns three views of the same data: a flat sections list, the Markdown export, and a versioned json_data document (schema_version: 2). The PDF is rendered with PyMuPDF (already a dependency). The report covers investigation id and question, documents, claims, conflicts, evidence for/against, competing hypotheses, verification results, the conclusion, confidence and its interpretation, missing evidence and unresolved questions, human-review status, an agent-activity summary, limitations and the audit trail. Guarantees include source traceability, observed-vs-inferred labelling, no fabricated evidence and HTML escaping of untrusted text.

16. Knowledge Graph

build_knowledge_graph reads the same records the agents reason over and returns typed nodes (document, claim, evidence, conflict, hypothesis, conclusion, agent, investigation) and typed edges. Nothing is invented: every node is a stored row or a registered agent, and every edge is backed by a persisted foreign key, association row, JSON id list, or ledger/event row. Edges carry a verified flag distinguishing direct persisted links from inferred ones. The frontend renders it with React Flow: filters with live counts, a verified/inferred legend, a node inspector, and an evidence-path highlight.

17. Optional AI-assisted reasoning & safety

The LLM layer is a complement, never a dependency. services/llm_service.py defines an LLMProvider ABC and an OpenAICompatibleProvider. Two layers control activation: a deployment switch (ENABLE_LLM + a key make it *available*) and a user preference (a persisted reasoning_mode of deterministic or ai_assisted). Safety properties, enforced in code and tested:

- Off by default; no key required. Without a provider, AI-assisted mode degrades gracefully to deterministic and reports why.
- Documents are data, not instructions. Text is wrapped in data delimiters and the system prompt forbids following embedded instructions.
- No fabricated evidence or citations. Supporting quotes must appear verbatim in the source; citations must resolve to real evidence rows.
- Transparent provenance. LLM-derived items are tagged ai_suggested.
- Resilient to provider failures. Timeouts, rate limits, 5xx and malformed responses are retried, then fall back to deterministic behaviour.
- Keys never leak. The key is only sent in the Authorization header and is never logged or returned.

18. Security, configuration & secrets

- Upload validation, safe filenames and a path-traversal guard; uploaded files are never executed.
- Optional bearer-token gating on every /api route (except health); proven by an exhaustive unauthenticated-access test.
- Secrets are environment-driven only (API_TOKEN, OPENAI_API_KEY); .env and

databases are git-ignored; a logging filter redacts every configured secret.

- Errors never return stack traces or secrets.
- Configuration is fully environment-driven (database URL, CORS origins, upload limit, environment, upload dir, log level/format, docs toggle, trusted hosts, API token, scheduler weights, iteration/evidence defaults, LLM settings).

---

19. Technology stack

Layer	Technology
Backend	FastAPI, SQLAlchemy 2.0, Pydantic v2, SQLite, Uvicorn
Extraction	PyMuPDF (PDF), python-docx (DOCX), rule-based NLP for text
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, React Flow, Recharts
Realtime	Server-Sent Events (SSE)
Testing	pytest (backend), TypeScript build (frontend)
Packaging	Docker, Docker Compose, Caddy (production TLS)

---

20. REST API surface

Base path /api. Highlights:

- Health & config: GET /health, GET /health/live, GET /config, GET/PUT /settings/reasoning.
- Documents: POST /documents/upload, GET /documents, GET /documents/{id}, POST /documents/{id}/reprocess, DELETE /documents/{id}.
- Claims: GET /claims, GET /claims/{id}.
- Conflicts: GET /conflicts, GET /conflicts/{id}, GET /conflicts/{id}/hypotheses/ranking, GET /conflicts/{id}/evidence-gaps, GET /conflicts/{id}/reviews, PATCH /conflicts/{id}.
- Evidence: GET/POST /conflicts/{id}/evidence, GET /evidence.
- Hypotheses: GET/POST /conflicts/{id}/hypotheses, GET /hypotheses, PATCH /hypotheses/{id}.
- Reviews: POST /conclusions/{id}/review, GET /conclusions/{id}/reviews, GET /conflicts/{id}/reviews, GET /reviews.
- Investigations: create/list/get/update, start, pause, resume, cancel, activity, report, report/export, and the events SSE stream.
- Blackboard: GET /blackboard, GET /blackboard/snapshot (with days), /blackboard/events, and trace endpoints.
- Knowledge graph: GET /knowledge-graph.
- Agents: GET /agents. Audit: GET /audit. Search: GET /search.
- Demo: POST /demo/load, POST /demo/reset.

List endpoints return a paginated envelope { items, total, page, page_size }; errors return { detail, code }.

---

21. Frontend application

Single-page React application (React Router) with pages for:

- Overview / Dashboard - live counts, charts, distributions and filter cards.
- Conflicts and Conflict detail - flagged explanations, side-by-side claims, evidence, hypotheses, review actions.
- Evidence - global evidence list with provenance and the new assessment

panel.

- Hypotheses, Investigations and the Investigation workspace - live agent activity, event stream, progress.
- Knowledge Graph, Reports, Audit Trail, Agents, Settings, and document Browse/Detail.

The typed API client mirrors the backend contract; tsc -b && vite build is the authoritative gate.

---

22. Testing strategy

- Backend: 216 tests in app/tests/ covering extraction, conflict detection, all six agents, the scheduler, orchestration, the blackboard, the SSE progress stream, hypothesis ranking, the audit trail, human review, multi-factor evidence assessment, dashboard snapshot filters/distributions, reports, the knowledge graph, the demo scenario, the optional LLM layer (mocked provider), security, settings, and the HTTP API.
- Isolation: tests use a temporary database and temporary uploads; no network or API key is required.
- Determinism: a run_investigation_sync fixture runs investigations to completion synchronously so agent-output tests do not race the background thread.
- Frontend: TypeScript type-check + production build.
- Evaluation harness: a curated labelled dataset measures detection quality.

Run the backend suite:

```
cd neurosync/backend
.\venv\Scripts\python.exe -m pytest app\tests -q
```

---

23. Evaluation results

Deterministic mode, LLM disabled, isolated temporary database. Conflict detection (positive class = confirmed contradiction; 32 examples):

Metric	Value
Precision	0.9091
Recall	0.8333
F1	0.8696
False-positive rate	0.0500
False-negative rate	0.1667
Accuracy	0.9062
Confusion	TP=10, FP=1, FN=2, TN=19

Evidence sufficiency (6 scenarios): status accuracy 1.0000 (6/6), partition traceability 1.0000. Conclusion traceability (demo investigation): 7/7 fully traceable (rate 1.0000).

Known failure cases (3, retained deliberately, all lexical_variation_and_negation): lex_subject_synonym_temp and lex_state_synonym_running (false negatives) and lex_negation_scope_false_positive (false positive). These are documented limitations of the deterministic detector, not hidden.

---

24. Run instructions (local)

Prerequisites: Python 3.11+ and Node.js 18+.

Backend (terminal 1):

```
cd neurosync/backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

API at http://127.0.0.1:8000 (interactive docs at /docs).

Frontend (terminal 2):

```
cd neurosync/frontend
npm install
npm run dev -- --host 127.0.0.1
```

Open http://127.0.0.1:5173. The dev server proxies /api to the backend.

Optional checks:

```
cd neurosync/backend
.\venv\Scripts\python.exe -m pytest app/tests -q
.\venv\Scripts\python.exe -m evaluation.run_evaluation
```

See it work in 30 seconds: open the app -> Overview -> Load demo -> Start -> watch the agents work in real time -> open Conflicts -> open Reports.

---

25. Docker & deployment

Development stack:

```
cd neurosync
docker compose up --build
```

Production (single host, Caddy terminates TLS, backend has no public port, API token injected server-side):

```
cp .env.prod.example .env     # fill in SITE_ADDRESS, ACME_EMAIL, CORS, API_TOKEN
docker compose -f docker-compose.prod.yml up -d --build
```

Backups via scripts/backup.sh (and scripts/restore.sh). Deployment is a manual operator action; see docs/deployment.md.

---

26. Known limitations

1. Lexical variation & negation - the deterministic detector can miss contradictions expressed with different wording and can over-flag a mis-scoped negation (measured: precision 0.9091 / recall 0.8333).
2. Knowledge Graph is O(total records) - built in memory by scanning all records; latency grows with corpus size (~443 ms for the demo corpus).
3. Single host, single process - the scheduler and event bus are in-process and SQLite allows one writer; do not horizontally scale on SQLite.
4. No user accounts (single-tenant) - a single shared bearer token; restrict access at the edge.
5. Optional AI layer is off by default and not live-tested (provider mocked in tests).
6. No standalone frontend test runner - correctness covered by TypeScript build.
7. Schema evolution without a migration framework - additive migrations only.

8. Small synthetic evaluation dataset - metrics are contract checks, not a general accuracy claim.
9. Production path not executed in the verification environment (Docker not installed there); configs reviewed.

---

27. Anticipated external questions & answers

Q1. What exactly is NeuroSync? A full-stack, evidence-driven investigation workspace. It ingests documents, extracts structured claims, detects contradictions across sources, and lets six specialised agents investigate those conflicts over a shared blackboard, with full traceability and human review.

Q2. What is the core innovation? A deterministic-by-default, auditable multi-agent blackboard system with an explainable dynamic scheduler, uncertainty as a first-class output, and safety-validated optional AI. It is not a chatbot; it is an inspectable reasoning engine.

Q3. How is this different from an LLM chatbot over documents? (1) Every step is deterministic and reproducible without any model; (2) reasoning is decomposed into specialised agents coordinated by a scored scheduler, not one prompt; (3) decisions are backed by structured evidence with provenance; (4) the system explicitly reports uncertainty and refuses to force conclusions; (5) a full audit trail attributes every change.

Q4. Does it need an API key or internet? No. It runs fully offline in deterministic mode. AI is an optional accelerator enabled in Settings.

Q5. How do you prevent the AI from hallucinating? Document text is passed as data with prompt-injection hardening; supporting quotes must appear verbatim in the source; any citation that does not resolve to a real evidence row is discarded; and LLM-derived items are tagged ai_suggested.

Q6. How accurate is conflict detection? On the curated 32-example dataset: precision 0.9091, recall 0.8333, F1 0.8696. These are contract checks on a small synthetic set, not a general accuracy claim; the three known failure cases are documented.

Q7. Is it scalable? It is designed for single-host, single-tenant use. The runtime and event bus are in-process and SQLite is a single-writer store, so scale vertically, not horizontally. A queue + Postgres migration is the documented path beyond that.

Q8. How is security handled? Bearer-token gating on all API routes, upload validation and path-traversal guards, environment-only secrets, log redaction, and safe error responses. No user accounts (intentional single-tenant design).

Q9. What technologies are used? FastAPI, SQLAlchemy 2.0, Pydantic v2, SQLite on the backend; React 19, TypeScript, Vite and Tailwind on the frontend; SSE for realtime; pytest for tests; Docker/Caddy for production.

Q10. How do you know it works? A 216-test automated suite, a green production build, a labelled evaluation harness with reproducible metrics, an end-to-end smoke test, and a documented release verification. Nothing is asserted without a measurement.

Q11. How do I run it? Two terminals: run the FastAPI backend with uvicorn app.main:app --reload --port 8000, and the frontend with npm run dev. Then Load demo and Start.

Q12. What is the "blackboard"? The single shared knowledge store (documents, claims, conflicts, evidence, hypotheses, conclusions, activity, events) that all agents read from and write to. It is the only shared state.

Q13. Why a dynamic scheduler instead of a fixed pipeline? So the system adapts to the data: each cycle it scores all available actions by severity, uncertainty, evidence gap and information gain, and runs the most valuable one - and logs why.

Q14. How does the system handle uncertainty? It surfaces missing information, unresolved questions and limitations explicitly, and can return inconclusive rather than forcing an answer. Evidence items are assessed on multiple factors with an explicit "unknown" label where data is missing.

Q15. Can a human override the system? Yes. Reviewers can approve, reject, mark inconclusive, request more investigation, or reopen a conflict; every decision is timestamped, attributed and recorded in the audit trail.

Q16. What are the main limitations? Lexical-variation misses in deterministic detection; O(records) knowledge-graph build; single-host/single-worker design; no user accounts; optional AI off by default; no frontend unit-test runner; additive-migrations-only schema evolution; small synthetic evaluation dataset.

Q17. Is there a demo? Yes - a clearly labelled "Industrial Equipment Diagnostic Investigation" (machine M-17 temperature disagreement) is loadable from the Overview page and always marked as demonstration data.

Q18. Where is the documentation? docs/ARCHITECTURE.md, docs/API_REFERENCE.md, docs/TESTING.md, docs/FEATURE_GAP_ANALYSIS.md, plus docs/orchestration.md, docs/deployment.md, docs/performance.md, docs/user_guide.md, docs/release_notes.md, docs/release_checklist.md and docs/known_issues.md.

Q19. What was added in the latest upgrade? Multi-factor evidence assessment, review decisions inconclusive/reopen, dashboard filters/distributions with deep links, and the requested documentation set - all verified (216 tests passing; smoke test green).

Q20. What would you build next? A synonym/ontology layer for deterministic recall, a persistent incremental graph projection, a queue + Postgres backend for horizontal scale, an optional multi-user model, and a Vitest/Playwright frontend test suite.

---

28. Glossary

- Blackboard: the shared knowledge store all agents read/write.
- Claim: a structured assertion extracted from a document.
- Conflict: a detected contradiction between two claims.
- Evidence: anything that bears on a conflict, with origin and reliability.
- Hypothesis: a candidate explanation for a conflict.
- Conclusion: the Decision Agent's final assessment.
- Scheduler: scores candidate actions and picks the most valuable next step.
- Support indicator: a bounded score (not a calibrated probability).
- SSE: Server-Sent Events, used for the live investigation stream.

End of report. NeuroSync v1.0.0 - provided as a reference implementation for demonstration and evaluation.